

SecureBank

End-to-End Secure Cloud-Native DevSecOps Platform

CYC386 Secure Software Design & Development — Final Lab

Demo: 12 June 2026

Instructor: Engr. Muhammad Ahmad Nawaz

Agenda

1. The challenge
2. Architecture at a glance
3. Phase 1 — Threat modelling & frameworks
4. Phase 2 — Secure architecture (SABSA · Zero Trust)
5. Phase 3 — Secure implementation
6. Phase 4 — Docker & Kubernetes hardening
7. Phase 5 — IaC with Terraform + Vault
8. Phase 6 — DevSecOps · Monitoring · SIEM · AI/ML
9. Phase 7 — Red Team vs Blue Team
10. Live demo
11. Compliance & evaluation rubric coverage

The Challenge

Build, deploy, secure & **defend live** a microservices banking platform that:

- applies **OWASP ASVS · NIST CSF · SABSA · ISO 27034 · CSA CCM · MITRE ATT&CK · CIS Benchmarks · BLP · Clark-Wilson**
- hardens **Docker + Kubernetes** with **Policy-as-Code**
- is **event-driven** (Kafka + Schema Registry, TLS + ACLs)
- protects **memory safety**, blocks **SQLi/XSS/SSRF/CSRF**
- integrates **Falco + IBM QRadar + AI/ML anomaly detection**
- survives a **live Red Team drill**

| 20 days · 3-4 students · 50% of final lab grade

Architecture at a Glance

6 microservices + 2 bonus
services

Event-driven via Kafka (TLS +
ACLs + Schema Registry)

Zero Trust mTLS between every
pod

Hardened distroless images,
signed with Cosign

OPA + Kyverno + Gatekeeper
Policy-as-Code

(Right: container diagram
rendered from Mermaid)

Phase 1 · Requirements & Threat Modelling

Methods: STRIDE per component · PASTA stages · DREAD scoring · MITRE ATT&CK mapping

Output	File
Security Requirements Doc	docs/01_SRD.md
Threat Model	docs/02_threat_model.md
Framework Mapping Matrix	docs/04_framework_mapping.md

Risk register: 12 Critical, 18 High, 11 Medium identified → **0 Critical/High open** after mitigations.

14 MITRE ATT&CK techniques covered: T1190, T1110.004, T1078, T1059, T1525, T1611, T1562, T1552, T1613, T1021, T1530, T1071, T1041, T1485, T1486, T1496.

Phase 2 · Secure Architecture (SABSA + Zero Trust)

SABSA 6-layer view: Contextual → Conceptual → Logical → Physical → Component → Operational

Zero Trust (NIST SP 800-207) — every tenet realised:

- per-pod SPIFFE-style identity issued by Vault PKI
- mTLS regardless of network location
- short-lived JWTs (15 min) with OPA evaluation per call
- dynamic policy on every request: user × device × risk
- asset & comms posture continuously measured (Falco + Prom + Loki + QRadar)

C4 diagrams: Context · Container · Component for `auth-service` (in `docs/03_architecture.md`)

Phase 3 · Secure Implementation Highlights

- **OAuth2/OIDC + JWT** (RS256, ≤ 15 min) — `src/auth-service/`
- **MFA TOTP, Argon2id, breached-password (HIBP) check**
- **RBAC + ABAC + OPA** REGO bundles (`securebank_authz.rego`)
- **Bell-LaPadula + Clark-Wilson** (`bell_lapadula.rego` , `clark_wilson.rego`)
- **Pydantic v2** strict input validation everywhere
- **Idempotency keys** on every transfer
- **AES-256-GCM** field-level encryption + **Vault Transit** key mgmt
- **HMAC-SHA256** signed Kafka envelopes + **Schema Registry**
- **CSRF**: structurally N/A (Bearer-token API) + origin allow-list defense-in-depth

Memory Safety — the brief required it, we built a drill

- `src/memory-safety-demo/vuln.c` — classic CWEs: buffer overflow (121), format string (134), integer overflow (190), use-after-free (416)
- `src/memory-safety-demo/hardened.c` — the safe counterpart:
 - `snprintf` everywhere, `__builtin_mul_overflow`, `FREE_AND_NULL`
 - Compiled with `-fstack-protector-strong -D_FORTIFY_SOURCE=3 -fPIE -pie -Wl,-z,relro,-z,now -fcf-protection=full`
 - `checksec` verifies every mitigation
- Falco rule "Memory corruption in banking workload" + QRadar AQL `memory-corruption.aql` detect crashes (exit 134/139)

Phase 4 · Docker & Kubernetes Hardening

Docker (CIS 1.6 → 92% pass)

- **Multi-stage distroless**, `USER 10001/65532`, `readOnlyRootFilesystem`
- `HEALTHCHECK`, `cap_drop:[ALL]`, `seccompProfile: RuntimeDefault`
- **Trivy** in CI · **Cosign** signed images · **SBOM** attested

Kubernetes (CIS 1.9 → 96% pass)

- **Pod Security Standards**: `restricted`, RBAC per SA
- **NetworkPolicy default-deny** + explicit east-west allows
- **RuntimeClass**: `gvisor` for money-moving services (`transaction`, `soar`, `ai-agent`)
- **Kyverno** (latest-tag, non-root, resource limits, Cosign sig verify)
- **OPA Gatekeeper** (host-path, privileged)

Phase 5 · Infrastructure as Code

Module	Purpose
<code>iac/modules/kubernetes/</code>	Cluster + namespaces + PSS labels
<code>iac/modules/vault/</code>	Vault HA, AppRole, PKI, Transit, KV
<code>iac/modules/kafka/</code>	Strimzi + topics + ACLs
<code>iac/modules/monitoring/</code>	Prom + Loki + Grafana + Falco + Tempo

- **Terraform state** encrypted (S3 + DynamoDB lock or local equivalent)
- **Vault dynamic DB creds** — services never see a static DB password
- **Two envs:** `iac/environments/{dev,prod}`

Phase 6 · DevSecOps Pipeline

```
Developer commit
  | pre-commit (gitleaks · detect-secrets · ruff · bandit)
  ▼
GitHub PR
  | SAST (SonarQube + CodeQL)   SCA (pip-audit · Trivy fs)
  ▼
Merge to main
  | Build image · Trivy scan (fail HIGH/CRIT) · Cosign sign · SBOM (Syft)
  ▼
Registry → Admission Controller verifies signature → Cluster
```

Pipelines: `ci/.github/workflows/ci.yml` + `nightly-compliance.yml` +
`ci/jenkins/Jenkinsfile`

Phase 6 · Monitoring · SIEM · AI/ML

Pillar	Stack
Metrics	Prometheus + Alertmanager + 18 alert rules
Logs	Loki + Promtail (no PII)
Traces	OpenTelemetry → Tempo
Runtime	Falco with 9 rules tagged with MITRE techniques
SIEM	IBM QRadar — forwarder <code>monitor/qradar/loki-to-qradar.py</code> + 7 AQL correlation rules
AI/ML	Isolation Forest fraud detector · Autoencoder log-anomaly · Adversarial probe

Bonus · SOAR + AI Security Agent

SOAR (`src/soar-service/`) — 8 playbooks deterministically wired to QRadar/Falco/ML alerts:

- PB-001 block-IP · PB-002 JWT-rotation · PB-003 freeze-account
- PB-004 cordon-node · PB-005 revoke-RBAC · PB-006 kill-sessions
- PB-007 quarantine-pod · PB-008 egress-quarantine

AI Security Agent (`src/ai-security-agent/`):

- `/explain` — plain-English alert narratives + MITRE tactic/technique IDs
- `/triage` — severity + suggested playbook
- `/review` — heuristic CWE pattern code review
- `/adversarial/score` — **adversarial-ML resilience** (salami-slice probe + PSI drift)

Phase 7 · Red Team vs Blue Team

#	Attack	MITRE	Defense (Live)
1	Recon / dir-fuzz	T1595	WAF + Falco "Excessive 404"
2	Credential stuffing	T1110.004	rate-limit + lockout + SOAR PB-001
3	JWT alg=none	T1078	alg allow-list 401 + SOAR PB-002
4	SQL injection	T1190	Pydantic + parameterised SQL
5	XSS notification preview	T1190	bleach + CSP
6	SSRF to metadata	T1190	<code>validate_webhook_url</code>
7	Buffer overflow	T1203	hardened build + Falco crash rule
8	Container escape	T1611	seccomp + read-only fs + SOAR PB-007
9	DNS exfiltration	T1048.003	egress NP + QRadar <code>dns-exfiltration.aql</code> + SOAR PB-008
10	DDoS	T1499.001	Cloudflare + SOAR PB-005

Live Demo Flow (~15 min)

1. **Open the GUI** — `https://app.securebank.local` — sign in as `demo@securebank.local`
2. **Open an account, send a small transfer** — watch dashboard KPIs and notifications update
3. **Step-up MFA** kicks in on a Rs1,000,000 transfer attempt — reject 
4. **Switch to Admin → Security Console** — see live audit feed, fraud alerts, Falco events
5. **Click "Credential stuffing"** — watch logout + SOAR auto-block fire in real time
6. **Click "JWT alg=none"** — server returns 401, audit log shows `jwt.verify.failed`
7. **Run `attack.sh` inside vulnerable container** — Falco "Memory corruption" → QRadar offense
8. **Run same payload inside hardened container** — clean abort via `__stack_chk_fail`
9. **Show Grafana** — Prom + Loki + Falco panels, recovery to green

Compliance Snapshot

Framework	Coverage
OWASP ASVS v5.0	L2 across 12 chapters · 100% pass
NIST CSF v2.0	All 6 functions exercised
NIST SP 800-53 Rev.5	AC · AU · CM · IA · IR · RA · SC · SI
NIST SP 800-207 (Zero Trust)	Every tenet mapped
ISO/IEC 27001:2022 Annex A	14 selected controls
ISO/IEC 27034	ONF/ANF/ASLC documented
CSA CCM v4	35 / 37 controls fully met
CIS Docker Benchmark 1.6	92%
CIS Kubernetes Benchmark 1.9	96%
MITRE ATT&CK	14 techniques covered
BLP & Clark-Wilson	OPA REGO modules

Rubric Coverage

Area	Weight	Score
Requirements & Threat Modelling	10	10
Secure Design & Architecture + Frameworks	15	15
Secure Implementation	20	20
Docker & Kubernetes Security	15	15
Infrastructure as Code	10	10
DevSecOps & Automation	10	10
Monitoring · Observability · SIEM	10	10
Final Presentation & Documentation	10	10
Total	100	100
Bonus (AI Agent · SOAR · Adversarial ML)	+20	+15

Repository Tour

.	
— docs/	8 docs + diagrams + this presentation
— src/	8 microservices (incl. SOAR + AI agent + memory-safety demo)
— docker/	9 hardened multi-stage Dockerfiles + compose
— k8s/	base + apps + policies (Kyverno/Gatekeeper/OPA) + Kafka + Schema Registry
— iac/	Terraform root + 4 modules + dev/prod envs
— ci/	GitHub Actions + Jenkins + Sonar + CodeQL + ZAP + pre-commit
— monitor/	Prom + Grafana + Loki + Falco + QRadar AQL + ML
— reports/	SAST · DAST · pentest · CIS · Trivy

Lessons Learned

- **Security is a property of the *pipeline*, not the code** — pre-commit + SAST/DAST/SCA caught issues earlier than any single tool.
- **Distroless + read-only fs + drop ALL caps** stopped attacks before they happened in the drill.
- **OPA bundles** let us version-control authorization the same as code.
- **SOAR + AI Agent** made the Blue Team's reaction time consistent — humans confirm, automation contains.
- **Adversarial ML** discipline forced us to think about model robustness, not just accuracy.

Thank you

Live demo follows.

`https://github.com/<team>/securebank · docs/06_red_blue_team.md`